Software Bug Tracking System — D1

Team: Michael Nguyen, Ahmad Awaidah, Angelo Mustafa, Dev Patel

Client: Ahmad Awaidah, playing the role of an engineering manager at a software company

Date: November 9, 2026

Section 1: Client Brief

Client Speaking:

“We are a software company that builds and maintains several applications for our customers. Since our IPO, the company has grown quickly, and our engineers and IT staff can no longer keep track of bugs through email and spreadsheets. Instead of hiring more engineers, we want to outsource an internal tool: a Software Bug Tracking System. We will need to keep track of our projects and the bugs reported in each one, with every bug labeled by severity, status, and tags for what it affects, such as UI, security, or performance. We will also need to know which developers are assigned to each bug and in what role, the comments developers leave on bugs, and the software releases for each project. Finally, we will need reports showing unresolved bugs, the number of bugs per project, each developer’s workload, and the most recently reported bugs.”

Section 2: Client Interview

- **Q1.** What information should the system store about a software project?
- **A1.** Each project should have a unique ID, name, and description.
- **Q2.** What information should be recorded when a bug is reported?
- **A2.** A unique bug ID, title, description, severity, status, creation date, and associated project.
- **Q3.** Can a project contain multiple bugs?
- **A3.** Yes. A project may have zero or more bugs, but each bug must belong to exactly one project.
- **Q4.** Can multiple developers work on the same bug?
- **A4.** Yes. A bug may be assigned to multiple developers, and a developer may work on multiple bugs.
- **Q5.** What information should be recorded about a developer?
- **A5.** A unique developer ID, name, and email address.
- **Q6.** What information should be recorded when a developer is assigned to a bug?

- **A6.** The developer, bug, assignment date, and assignment role
- **Q7.** Can bugs have multiple categories or tags?
- **A7.** Yes. A bug can have multiple tags, such as UI, Security, and Performance.
- **Q8.** Can developers leave comments on bugs?
- **A8.** Yes. A developer may post multiple comments, and each comment must belong to one bug and have one author.
- **Q9.** Does the team need to track software releases?
- **A9.** Yes. Each release should have a version number, release date, and associated project.
- **Q10.** What reports should the system generate?
- **A10.** Reports showing unresolved bugs, bug counts per project, developer workloads, and the most recently reported bugs.
- **Q11.** Can a bug exist without an assigned developer?
- **A11.** Yes. Newly reported bugs may remain unassigned until someone is available.
- **Q12.** What happens when a bug is resolved?
- **A12.** Its status changes to Resolved. The system should retain the bug and its assignment history.

Section 3: Requirement List

Data Requirements

- **R1.** The system shall store software projects, each identified by a unique `project_id`, with a name and description.
- **R2.** The system shall store developers, each identified by a unique `developer_id`, with a name and unique email address.
- **R3.** The system shall store bugs, each identified by a unique `bug_id`, with a title, description, severity, status, and creation date.
- **R4.** Each bug may have zero or more tags, such as Security, UI, or Performance.
- **R5.** The system shall record developer assignments to bugs, including the assignment date and role.
- **R6.** The system shall store comments, each identified by a unique `comment_id`, with its text and creation timestamp.
- **R7.** The system shall store software releases, each identified by a unique `release_id`, with a version number and release date.

Relationship and Constraint Requirements

- **R8.** Every bug must belong to exactly one project. A project may contain zero or more bugs. (1:N)
- **R9.** A developer may be assigned to zero or more bugs, and a bug may be assigned to zero or more developers. (M:N)
- **R10.** Each developer–bug assignment must record an assignment date and role. A developer–bug pair may have at most one assignment record.
- **R11.** Every comment must belong to exactly one bug. A bug may have zero or more comments. (1:N)
- **R12.** Every comment must be written by exactly one developer. A developer may write zero or more comments. (1:N)
- **R13.** Every release must belong to exactly one project. A project may have zero or more releases. (1:N)
- **R14.** Bug severity must be one of: Low, Medium, High, or Critical.
- **R15.** Bug status must be one of: Open, In Progress, or Resolved.
- **R16.** Each release version must be unique within its project, although different projects may use the same version number.

Query Requirements

- **R17.** The system shall list bugs together with their associated project names and assigned developers.
- **R18.** The system shall identify projects containing more than a specified number of bugs.
- **R19.** The system shall identify developers assigned to more than a specified number of unresolved bugs.
- **R20.** The system shall display the most recently created bugs for each project.
- **R21.** The system shall identify bugs that have never been assigned to a developer.
- **R22.** The system shall rank bugs by severity within each project.
- **R23.** The system shall allow a new bug to be recorded for an existing project.
- **R24.** The system shall support transferring an assignment from one developer to another as a single transaction.
- **R25.** The system shall provide a reusable database view containing unresolved bugs.

Appendix: Glossary

Term	Definition
Project	A software application or development effort containing reported bugs.
Developer	A team member responsible for investigating, fixing, or discussing bugs.
Bug	A reported defect or unexpected behavior in a software project.

Term	Definition
Assignment	A relationship connecting a developer to a bug, including assignment information.
Severity	A classification indicating the seriousness of a bug.
Status	The current stage of a bug's progress.
Comment	A written message associated with a bug and authored by a developer.
Release	A specific version of a software project.
Tag	A label used to categorize a bug.
Unresolved Bug	A bug whose status is Open or In Progress.

How This Satisfies the Database Design Requirements

Requirement	Our Design
5–8 Entity Sets	Project, Developer, Bug, Assignment, Comment, Release (6, treating Assignment as associative)
At Least One 1:N	Project → Bug
At Least One M:N	Developer ↔ Bug
Multivalued Attribute	Bug `{tag}`
Relationship Attribute	Assignment `assigned_at`, `role`
Primary Keys	Unique IDs for major entities
Participation Constraints	Explicitly stated in R8–R13
Concrete Queries	R17–R25

Scope & Assumptions

- The system will focus on storing and querying software bug information rather than providing a complete project management platform.
- The initial implementation will not include user authentication, automated bug detection, email notifications, GitHub integration, file attachments, or deployment to a public server.
- The system will use MySQL 8 for database storage and a lightweight Python Flask application for its user interface.